Podsumowanie Architektury i Implementacji Systemu

Poniższe zestawienie opisuje kluczowe funkcje i klasy zaimplementowane w systemie detekcji zmian skórnych, z podziałem na moduły przetwarzania danych, modelowania oraz ewaluacji.

1. Moduł Przetwarzania i Ładowania Danych (`src/data_loader.py`)

- `create_generators(...)`: Funkcja odpowiedzialna za inicjalizację potoków danych. Wykorzystuje `ImageDataGenerator.fl` do wczytywania obrazów z podziałem na zbiory: treningowy, walidacyjny i testowy. Automatycznie aplikuje funkcję `preprocess_input` oraz techniki augmentacji danych (np. rotacje, przesunięcia), co zapewnia spójność danych wejściowych z wymaganiami modelu *EfficientNetB0*.

2. Architektura Modelu (`src/model.py`)

- `create_model(num_classes, input_shape, train_base)`: Konstruktor architektury sieciowej. Składa się z:
  - Szkieletu (*backbone*) **EfficientNetB0** z wagami *ImageNet* (bez warstw wierzchnich).
  - Warstwy **GlobalAveragePooling2D**, konwertującej mapy cech na wektor.
  - Głowicy klasyfikacyjnej: warstwy **Dense(128)** z aktywacją ReLU oraz końcowej warstwy **Dense(num_classes)** z aktywacją *softmax*.
- **Atrybut** `model.base_model`: Referencja do szkieletu sieci, umożliwiająca selektywne "odmrażanie"wag podczas procesu *fine-tuningu*.

3. Proces Treningowy i Optymalizacja (`src/train.py`)

- `OversampledSequence`: Niestandardowa klasa dziedzicząca po `Keras.Sequence`. Rozwiązuje problem nierównowagi klas poprzez nadpróbkowanie (*oversampling*) klasy mniejszościowej (*Melanoma*) metodą losowania ze zwracaniem. Gwarantuje, że każda epoka zawiera zbalansowaną liczbę przykładów dla wszystkich kategorii.
- `categorical_focal_loss(gamma, alpha)`: Implementacja funkcji straty *Focal Loss*. W przeciwieństwie do standardowej entropii krzyżowej, skupia ona wagę uczenia na przykładach trudnych do sklasyfikowania oraz pozwala na dodatkowe wzmocnienie klasy krytycznej (*Melanoma*) poprzez parametr *alpha*.
- **Callbacki**:
  - `EarlyStopping`: Zapobiega przeuczeniu poprzez przerwanie treningu, gdy strata walidacyjna przestaje maleć.
  - `ReduceLROnPlateau`: Automatycznie zmniejsza tempo uczenia (LR), gdy model osiąga plateau.
  - `ModelCheckpoint`: Zapisuje stan modelu o najlepszych parametrach napotkanych podczas treningu.

4. Ewaluacja i Inferencja (`src/evaluate.py` & `src/predict.py`)

- `model.predict(...)`: Generuje rozkład prawdopodobieństwa dla każdej klasy.
- **Metryki *Scikit-learn***:
  - `classification_report`: Generuje szczegółowe statystyki *Precision*, *Recall* oraz *F1-score* dla każdej z trzech klas.
  - `confusion_matrix`: Macierz pomyłek pozwalająca zidentyfikować, które klasy są najczęściej mylone (np. *Melanoma* interpretowana jako znamię łagodne).
- `predict_single_image(...)`: Funkcja pomocnicza realizująca predykcję dla pojedynczego zdjęcia, zwracająca *Top-3* najbardziej prawdopodobnych klas wraz z ich pewnością (confidence).

Cykl życia modelu: od surowych danych do predykcji

Poniższy opis przedstawia przepływ danych i etapów przetwarzania w systemie, od momentu wczytania obrazu z dysku po wygenerowanie diagnozy klinicznej.

1. Przygotowanie danych (Data Ingestion)

Proces rozpoczyna się w strukturze katalogów `data/train/`, `data/validate/` oraz `data/test/`. Obrazy są organizowane w podfoldery odpowiadające poszczególnym jednostkom chorobowym. Za ładowanie odpowiada moduł `src/data_loader.py`, który:

- Wykorzystuje `ImageDataGenerator.flow_from_directory` do strumieniowego przesyłania obrazów.
- Stosuje funkcję `preprocess_input` dedykowaną dla architektury *EfficientNet*.
- Implementuje augmentacje (obroty, przesunięcia) w celu zwiększenia różnorodności zbioru treningowego.

2. Definicja i budowa modelu

Model jest budowany w `src/model.py` przy użyciu techniki *Transfer Learningu*:

- **Ekstraktor cech**: Wykorzystano model *EfficientNetB0* z wagami *ImageNet*.
- **Głowica (Top)**: Global Average Pooling oraz warstwy gęste (Dense) z aktywacją *Softmax* dla 3 klas.
- **Integracja**: Szkielet jest dostępny jako `model.base_model`, co pozwala na stabilne zarządzanie procesem zamrażania i odmrażania wag.

3. Strategia treningowa (Staged Training)

Proces uczenia w `src/train.py` jest podzielony na dwa kluczowe etapy:

1. **Trening głowicy**: Szkielet jest zamrożony. Model uczy się mapować cechy z *EfficientNet* na specyficzne klasy chorób skóry. Wykorzystywana jest technika **OversampledSequence**, która balansuje klasy poprzez nadpróbkowanie czerniaka (*Melanoma*).
2. **Dostrajanie (Fine-tuning)**: Szkielet zostaje częściowo odmrożony. Model jest ponownie kompilowany z bardzo niskim współczynnikiem uczenia ($LR = 10^{-5}$) i funkcją **Focal Loss**, która skupia wagę na trudnych, błędnie klasyfikowanych przykładach.

4. Ewaluacja i Inferencja

- **Ewaluacja**: Skrypt `src/evaluate.py` generuje deterministyczne wyniki na zbiorze testowym, tworząc macierz pomyłek oraz raport klasyfikacji (Precision, Recall, F1).
- **Predykcja**: Moduł `src/predict.py` ładuje finalny model `.h5` oraz mapowanie klas, zwracając trzy najbardziej prawdopodobne diagnozy (*Top-3 predictions*) wraz z ich poziomem ufności dla każdego nowego zdjęcia.

5. Artefakty wynikowe

W wyniku działania potoku generowane są następujące pliki:

- `best_model.h5` – plik binarny z wagami modelu o najwyższej czułości.
- `class_indices.json` – mapowanie indeksów numerycznych na nazwy chorób.
- `predictions_analysis_summary.json` – szczegółowa analiza błędów typu *false negative* i *false positive*.

Kluczowe Komponenty Architektury

1. **Transfer Learning (Wiedza Wstępna)**: Model bazuje na szkielecie *EfficientNetB0* z wagami *ImageNet*. Dzięki temu sieć posiada umiejętność ekstrakcji uniwersalnych cech wizualnych (krawędzie, tekstury, kształty) już od pierwszej epoki treningu.
2. **Dedykowana Głowica Klasyfikacyjna**: Nad szkieletem bazowym nadbudowano autorski moduł klasyfikacyjny:
   - *GlobalAveragePooling2D*: Redukuje wymiarowość map cech do wektora o stałej długości.
   - *Warstwa Dense (128 jednostek)*: Warstwa neuronowa z aktywacją ReLU, pełniąca rolę interpretatora cech.
   - *Warstwa Wyjściowa (Softmax)*: Zwraca rozkład prawdopodobieństwa dla 3 specyficznych klas diagnostycznych.

3. **Mechanizmy Balansowania Zbioru (Oversampling)**: W celu uniknięcia faworyzowania klas liczniejszych, zaimplementowano klasę `OversampledSequence`. Gwarantuje ona równomierny rozkład próbek w każdej partii danych (batch), co jest krytyczne dla wysokiej czułości (*Recall*) w wykrywaniu czerniaka.

4. **Zaawansowana Funkcja Straty: Focal Loss**: Zamiast standardowej entropii krzyżowej, model wykorzystuje funkcję *Categorical Focal Loss*, zdefiniowaną wzorem:

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t) \tag{1}$$

   Mechanizm ten wymusza na modelu skupienie się na przykładach trudnych i błędnie klasyfikowanych, jednocześnie redukując wpływ przykładów łatwych, których model jest już pewien.

5. **System Inteligentnego Nadzoru (Callbacki)**: Proces uczenia jest dynamicznie kontrolowany przez:
   - *EarlyStopping*: Przerywa trening w optymalnym momencie, zapobiegając przeuczeniu (*overfitting*).
   - *ReduceLROnPlateau*: Automatycznie zmniejsza tempo uczenia, gdy model przestaje poprawiać wyniki, co pozwala na precyzyjną optymalizację wag.
   - *ModelCheckpoint*: Zapewnia trwały zapis wyłącznie najlepszej iteracji modelu pod kątem monitorowanej metryki.